The Wayback Machine - https://web.archive.org/web/20251012034933/https://deploy-preview-2737--esphome.netlify.app/componen...

ESPHome

Table of Contents

- Viessmann Optolink
- Overview
- Configuration
- Platform configuration
- Configuration variables
- Sensors
- Configuration variables
- Explanation of `type`
- Binary Sensors
- Configuration variables
- Text Sensors
- Configuration variables
- Explanation of `type`
- Schedule plans
- Texts
- Configuration variables
- Numbers
- Configuration variables
- Switches
- Configuration variables
- Selects
- Configuration variables
- Troubleshooting
- IR-LED
- Too many sensors
- Logging of communication
- Viessmann unit reboot
- See Also

- Join the community
- Follow us on Twitter
- Source Code
- Contact (no support!)

ESPHome is part of the Open Home Foundation

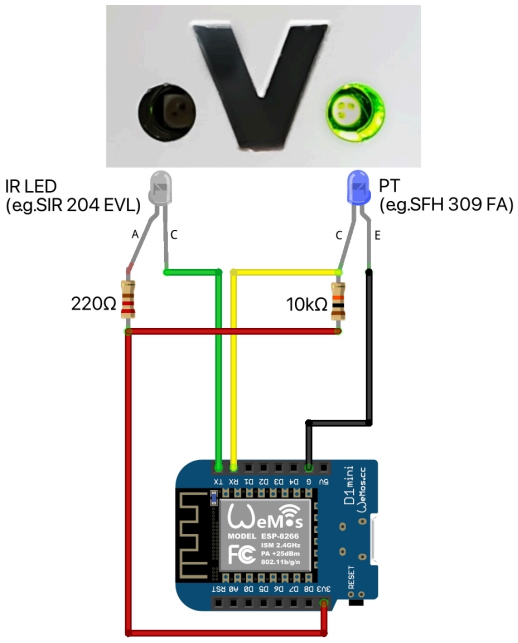
This site is powered by Netlify

Viessmann Optolink ¶

The `optolink` platform allows you to integrate Viessmann® heating component in ESPHome.

Using a simple DIY adapter cable you can read parameters of the heating component and also partly change them. The simplest version of the adapter cable consists of an IR diode, a phototransistor and two resistors, connected to the serial port pins of the ESP.

ESPHome is



Viessmann® Optolink connection with adapter cable.

Once configured, you can add further Sensors, Binary Sensors and Text Sensors to monitor parameters of the heating component. Moreover, you can add Switches, Numbers, Texts and Selects to control the heating component.

> **Note:**
>
> The `optolink` platform is based on the great work of many people in the OpenV project and the fantastic Arduino Library VitoWiFi of Bert Melis.

Overview

Viessmann® calls the parameters of the heating component datapoints. Each datapoint consists of a 32-bit address, the number of bytes, and optionally, a value factor by which the read value must be divided. The datapoints supported by your heating component depend on the built-in Vitotronic controlling unit. This page on the OpenV Wiki shows the supported units, this page the supported datapoints for all units (only in German language).

Configuration

Platform configuration

```yaml
# Example configuration entry
optolink:
  protocol: P300
```

Configuration variables

- **protocol** (**Required**, string): The protocol for communication (`KW` or `P300`; `GWG` is not supported). See this page
- **rx_pin** (**Required**, only ESP32, Pin): Receive pin (RX)
- **tx_pin** (**Required**, only ESP32, Pin): Transmit pin (TX)

- **logger** (**Required**, boolean): More detailled log output (Logger Component needed). See *Troubleshooting*. Defaults to 'false'.
- **communication_suspension** (*Optional*, Time): See *Troubleshooting*. Defaults to 20s
- **max_response_delay** (*Optional*, Time): The maximum duration between request and response before the platform suspects a communication interruption. Defaults to 2s

Warning:

The `optolink` platform uses UART0 for communication with the heating component. Don't mix it with other components that use UART0. Especially the Logger Component uses by default UART0 to write log messages to the serial port. If you plan to use the Logger Component, configure it to use another serial port.

```
logger:
  hardware_uart: UART1
```

Or deactivate the logging to UART0 with a baudrate value of 0. Even if you deactivate logging on UART0, the log outputs can be seen on the local computer via the *esphome* command line utility (Getting Started with the ESPHome Command Line).

```
logger:
  hardware_uart: UART0
  baud_rate: 0
```

Sensors ¶

```
# Example configuration entry
sensor:
  - platform: optolink
    name: Boiler Temperature
    address: 0xA309
    bytes: 2
    div_ratio: 100
    unit_of_measurement: °C
    device_class: temperature
    update_interval: 120s
```

Configuration variables ¶

- **address** (**Required**, hexadecimal): Address of datapoint
- **bytes** (**Required**, int): Number of bytes of datapoint
- **div_ratio** (*Optional*, int): Value factor of datapoint. Defaults to '1'.
- **type** (*Optional*, String): Sensor type (see table below). Defaults to 'DATAPOINT'
- **update_interval** (*Optional*, Time): The interval that the value should be checked.
- All other options from Sensor Component

Explanation of `type` ¶

type	sensor shows
`DATAPOINT` (Default)	value of the specified datapoint
`QUEUE_SIZE`	number of not yet processed datapoint requests

Binary Sensors

```yaml
# Example configuration entry
binary_sensor:
  - platform: optolink
    name: Disturbance
    address: 0x0A82
```

Configuration variables

- **address** (**Required**, hexadecimal): Address of datapoint
- **update_interval** (*Optional*, Time): The interval that the value should be checked.
- All other options from Binary Sensor Component

Text Sensors

```yaml
# Example configuration entries
text_sensor:
  - platform: optolink
    name: Error history 1
    address: 0x7590
    bytes: 9
    type: RAW
  - platform: optolink
    name: Heating operation mode
    address: 0x2323
    bytes: 1
    type: MAP
    filters:
      map:
        - "0 -> Off"
        - "1 -> Only hot water"
        - "2 -> Heating and hot water"
  - platform: optolink
    type: DEVICE_INFO
    name: Device Info
  - platform: optolink
    type: STATE_INFO
    name: Communication state
  - platform: optolink
    name: Heating schedule Monday
    type: DAY_SCHEDULE
    day_of_week: MONDAY
    address: 0x2000
    bytes: 56
```

Configuration variables

- **type** (**Required**, string): Type of text sensor (see table below)
- **address** (hexadecimal): Address of datapoint
- **bytes** (int): Number of bytes of datapoint

- **day_of_week** (string): Day of the week (only required for *type* DAY_SCHEDULE; see *Schedule plans*)
- **update_interval** (*Optional*, Time): The interval that the value should be checked.
- All other options from Text Sensor Component

Explanation of `type`

`type`	`address` Required	`bytes` Required	`day_of_week` Required	text sensor shows
`RAW`	yes	yes		the bytes read as raw characters
`MAP`	yes	yes		the bytes treated as a numerical value that can be mapped (see Text Sensor Component)
`DEVICE_INFO`				information of your Vitotronic controlling unit
`STATE_INFO`				state of communication
`DAY_SCHEDULE`	yes	yes	yes	schedule plan with On/Off time pairs

Schedule plans

The type `DAY_SCHEDULE` can be used to read daily schedule plans with On/Off time pairs. These time pairs are used for example, to turn on or off heating or warm water. Each day of the week can have up to four time pairs. The hour values are represented in 24h format, the minute values are multiples of `10`. For example, the following value means 'turn on for the time between 5:20 and 9:00 and between 16:00 and 21:30':

```
5:20 9:00 16:00 21:30
```

> **Note:**
>
> Currently only schedule plans configured by 56-byte datapoints are supported. Refer to the documentation of your Viessmann unit for suitable datapoints.

Texts

In contrast to a *text_sensor* of type *DAY_SCHEDULE* a *text* of this type allows you to not only read the schedule plan but also to modify it.

```yaml
# Example configuration entry
text:
  - platform: optolink
    name: Warm water schedule Monday
    address: 0x2100
    bytes: 56
    type: DAY_SCHEDULE
    day_of_week: MONDAY
    icon: mdi:shower
```

Configuration variables ¶

- **type** (**Required**, string): Type of text (currently only *DAY_SCHEDULE*)
- **address** (**Required**, hexadecimal): Address of datapoint
- **bytes** (**Required**, int): Number of bytes of datapoint (currently only *56*)
- **day_of_week** (string): Day of the week (see *Schedule plans* in section *text_sensor*)
- All other options from Text Component

Numbers ¶

```yaml
# Example configuration entry
number:
  - platform: optolink
    name: Room Temperature Setpoint
    unit_of_measurement: °C
    address: 0x2306
    bytes: 1
    min_value: 3
    max_value: 37
    step: 1
    mode: box
    icon: "mdi:home-thermometer"
    device_class: temperature
```

Configuration variables ¶

- **address** (**Required**, hexadecimal): Address of datapoint
- **bytes** (**Required**, int): Number of bytes of datapoint
- **div_ratio** (**Required**, int): Value factor of datapoint. Defaults to '1'.
- **min_value** (**Required**, float): Minimum possible value (Currently no negative values)
- **max_value** (**Required**, float): Maximum possible value (Currently no negative values)
- **step** (**Required**, float): Increment size
- **update_interval** (*Optional*, Time): The interval that the value should be checked.
- All other options from Number Component

Switches ¶

```yaml
# Example configuration entry
switch:
```

```
  - platform: optolink
    name: Economy mode
    address: 0x2302
    icon: mdi:sprout-outline
```

Configuration variables

- **address** (**Required**, hexadecimal): Address of datapoint
- All other options from Switch Component

Selects

```
# Example configuration entry
select:
  - platform: optolink
    name: Operation mode
    address: 0x2323
    bytes: 1
    map:
      - "0 -> Off"
      - "1 -> Only hot water"
      - "2 -> Heating and hot water"
```

Configuration variables

- **address** (**Required**, hexadecimal): Address of datapoint
- **bytes** (**Required**, int): Number of bytes of datapoint
- **div_ratio** (*Optional*, int): Value factor of datapoint. Defaults to '1'.
- **map** (**Required**, map): Mapping of numerical value to human readable value.
- **update_interval** (*Optional*, Time): The interval that the value should be checked.
- All other options from Select Component

Troubleshooting

If you encounter problems, here are some hints.

Warning:

The optical interface has a limited throughput, getting a response from the Viessmann controller takes a fragment of a second and is sequentially processed. Keep that in mind when defining sensors/components and define an appropriate value of `update_interval` for each of them. Otherwise the default update interval of 10 seconds will be used which can lead to unprocessed read and write requests.

Example config:

```
  - platform: optolink
    name: outdoor temperature
    address: 0x5525
    bytes: 2
    div_ratio: 10
    unit_of_measurement: °C
    device_class: temperature
    update_interval: 120s
```

IR-LED ¶

The IR LED is working in a frequency band outside of your eye's perception. To see the LED sending signals watch the LED through your smartphones camera. This way the IR light is visible and you can check if the LED is working.

Too many sensors ¶

If you configure a large number of datapoints you could run out of memory, especially on ESP8266 controllers. Try to avoid unnecesary components like the Web Server Component. The Debug Component, which is unfortunately itself a component with significant memory footprint, can help to check free memory.

Logging of communication ¶

If you configure the `optolink` platform with `logger: true`, the communication with the heating component is written to the console - provided you have configured the Logger Component with level `debug`.

Viessmann unit reboot ¶

In cases where the Viessmann device restarts (e.g. after a power outage) communication with the DIY adapter has to be stopped. Otherwise the boot process of the controlling unit will hang. The `optolink` platform suspends communication when it detects that there are no more responses to outstanding requests. The default time period of this suspension is 20s. You can overwrite this duration with the the parameter `communication_suspension` (see *Platform configuration*).

See Also ¶

- OpenV project
- VitoWiFi
- Edit this page on GitHub